Pac-Man

eslem yildirim

May 2026

Gameplay Video: https://www.youtube.com/shorts/QI0YOR$_4$86o

1 Introduction

This project is a Java-based implementation of the classic arcade game, Pac-Man. The main goal of the game is to navigate Pac-Man through a maze, eat all the pellets, and avoid the enemy ghosts. The project is built on a robust Object-Oriented Programming (OOP) architecture utilizing graph algorithms.

For rendering the graphics, the **StdDraw** library was used. StdDraw is a simple 2D drawing library. It handles the canvas creation, shape drawing, and keyboard inputs.

2 Game Mechanics

2.1 Core Mechanics

- **Collision Detection:** We detect collisions by checking the distance between the visual coordinates of Pac-Man and the ghosts. If the distance is smaller than a specific radius (e.g., 0.5), a collision happens, and the game state changes to LOST.
- **Input Buffering:** When the player presses a key, the game saves it as `requestedDirection`. Pac-Man only turns when there is no wall in that direction. This prevents Pac-Man from getting stuck on walls.
- **Animation Mechanics:** Animations are created by changing the visual state after a certain number of frames (using delay variables like `PACMAN_ANIMATION_DELAY`). For example, Pac-Man's mouth opens and closes every few frames.
- **State Transitions:** The game is controlled by a State Machine. The states are `START_SCREEN`, `READY`, `PLAYING`, `PAUSED`, `WON`, `LOST` (and `BACKTRACKING` for the state-rewind mechanic). The game logic only updates when the state is `PLAYING`.

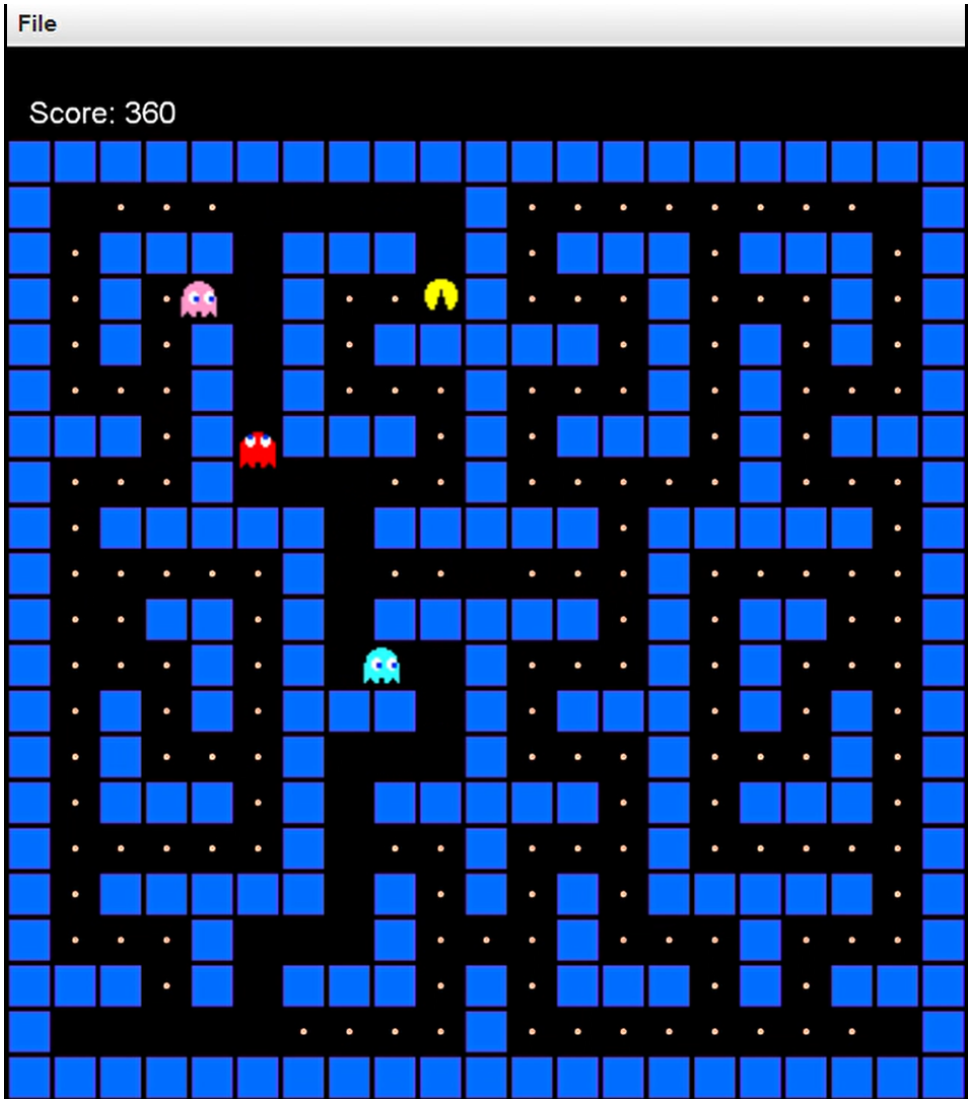


Figure 1: Gameplay screenshot showing Pac-Man and Ghosts.

2.2 Pathfinding and Target Selection

The ghosts use the **Breadth-First Search (BFS)** algorithm to find the shortest path to their targets. BFS explores the grid layer by layer, ensuring the shortest route is found in an unweighted grid.

Different ghost types have different personalities based on their `selectTarget()` methods:

- **Pinky:** Very aggressive. Targets Pac-Man's exact current position.
- **Blinky:** Moves towards specific corners or closest intersections.
- **Inky:** Moves randomly to confuse the player.

BFS Algorithm Snippet:

```
public Direction getNextMove(Position start, Position target,
    MapData mapData) {
     Queue<Node> queue = new LinkedList<>();
     boolean[][] visited = new boolean[rows][cols];

     queue.add(new Node(start.getRow(), start.getCol(), null,
        Direction.NONE));
     visited[start.getRow()][start.getCol()] = true;

     while (!queue.isEmpty()) {
          Node current = queue.poll();
          if (current.row == target.getRow() && current.col ==
             target.getCol()) {
               return getFirstStep(current); // Returns the first
                  move to take
          }
          // Add valid neighbors to queue
     }
     return Direction.NONE;
}
```

Pinky's Target Selection Snippet:

```
@Override
public Position selectTarget(Player player, MapData mapData) {
     // Pinky directly targets Pac-Man's position
     return player.getPos();
}
```

3 Technical Implementation

3.1 Data Structures and Algorithms

- **2D Arrays:** The maze map is stored as a 2D `char` array. It holds walls (`#`), pellets (`.`), and empty spaces.
- **Queue:** The standard Java `LinkedList` is used as a Queue for the BFS pathfinding algorithm. Queue uses First-In-First-Out (FIFO) logic, which is perfect for layer-by-layer searching.

3.2 OOP and Inheritance

The project heavily uses Inheritance and Polymorphism. We have a base class called `Enemy`. It contains common variables (position, speed) and methods (move, draw). The classes `Pinky`, `Blinky`, and `Inky` *extend* the `Enemy` class. They override the `selectTarget()` method to implement their unique behaviors without duplicating movement logic.

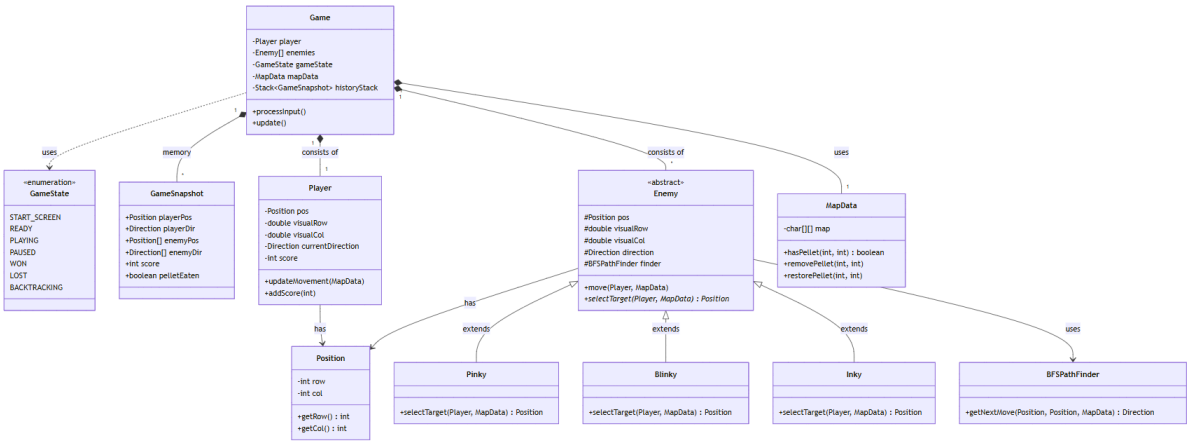


Figure 2: UML Diagram showing the Inheritance structure.

4 Advanced Mechanics: Time Backtracking

Rewind Feature Showcase: https://www.youtube.com/shorts/OLr1YXG75jM

As an experimental gameplay mechanic, a Time Backtracking (Rewind) system was implemented. If the player presses the 'R' key when the game is in the `WON` or `LOST` state, the game rewinds itself step by step until the starting point.

4.1 Data Structures Used

To achieve this, a **Stack** (Last-In-First-Out) data structure was built. Every time a character makes a logical move on the grid, or a pellet is eaten, a `GameSnapshot` object is created and pushed to the Stack.

4.2 Backtracking Logic

When the state becomes `BACKTRACKING`, the game pops a `GameSnapshot` from the Stack in every update cycle. The visual sliding is disabled, and the characters are teleported instantly to their exact grid coordinates stored in the snapshot. If a pellet was eaten in that snapshot, it is restored to the map, and the score decreases. This creates a fast, retro "rewind" effect.

```
1 // Saving the snapshot (Push)
2 if (logicalMove) {
3     historyStack.push(new GameSnapshot(player, enemies, score,
          pelletEaten, row, col));
4 }
5
6 // Rewinding the game (Pop)
7 if (gameState == GameState.BACKTRACKING && !historyStack.isEmpty
      ()) {
8     GameSnapshot past = historyStack.pop();
```

```
9       player.setPos(past.playerPos); // Teleport back
10      if (past.pelletEaten) {
11          mapData.restorePellet(past.eatenRow, past.eatenCol);
12      }
13  }
```